

Aula modelo

Disciplina: Programação aplicada à engenharia cartográfica

Maurício C. M. de Paulo - D.Sc.

2 de junho de 2026



- 1 Conceitos
- 2 Exercícios
- 3 Exercícios com EDGV



1 Conceitos

2 Exercícios

3 Exercícios com EDGV

Numéricos

- SMALLINT
- INTEGER
- BIGINT
- REAL
- DOUBLE PRECISION
- NUMERIC

Texto

- TEXT
- VARCHAR
- CHAR

Data e Hora

- DATE
- TIME
- TIMESTAMP
- TIMESTAMPTZ

Booleano

- BOOLEAN

Auto Incremento

- SERIAL
- BIGSERIAL

Estruturados

- JSON
- JSONB
- XML

Binários

- BYTEA

PostGIS

- geometry
- geography

```
CREATE TABLE exemplo (  
    id SERIAL PRIMARY KEY,  
    nome TEXT,  
    atributos JSONB,  
    geom geometry(Point,4326)  
);
```

Objetivo

Armazenar e consultar datas em uma tabela PostgreSQL.

Tabela de eventos

```
CREATE TABLE eventos (  
    id SERIAL PRIMARY KEY,  
    descricao TEXT,  
    data_evento DATE  
);  
  
INSERT INTO eventos  
(descricao, data_evento)  
VALUES  
( 'Prova 1', '2025-09-15'),  
( 'Entrega Trabalho', '2025-10-01'),  
( 'Prova Final', '2025-12-10');
```

Consultar eventos após 01/10/2025

```
SELECT *  
FROM eventos  
WHERE data_evento >= '2025-10-01';
```



Objetivo

Representar períodos utilizando um único campo.

Tabela de férias

```
CREATE TABLE ferias (  
    id SERIAL PRIMARY KEY,  
    funcionario TEXT,  
    periodo DATERANGE  
);  
  
INSERT INTO ferias  
(funcionario, periodo)  
VALUES  
(  
    'João',  
    '[2025-01-10,2025-01-25]'),  
  
(  
    'Maria',  
    '[2025-07-01,2025-07-20]');
```

Quem estava de férias em 15/01/2025?

```
SELECT funcionario
```

Exemplo: Liste todas as regiões existentes na tabela, sem repetir valores.

Dados

```
CREATE TABLE cidades (  
    id SERIAL PRIMARY KEY,  
    nome TEXT,  
    regiao TEXT  
);  
  
INSERT INTO cidades (nome, regiao)  
VALUES  
('Rio de Janeiro', 'Sudeste'),  
('São Paulo', 'Sudeste'),  
('Campinas', 'Sudeste'),  
('Curitiba', 'Sul'),  
('Recife', 'Nordeste');
```

Consulta

```
SELECT DISTINCT regiao  
FROM cidades;
```

Resultado

```
Sudeste  
Sul  
Nordeste
```

Observação

O comando DISTINCT elimina linhas duplicadas do resultado da consulta.



Conceito

Uma chave primária identifica unicamente cada registro de uma tabela.

Características

- Não pode ter valores repetidos
- Não pode ser nula
- Cada linha possui uma identificação única

Exemplo

```
CREATE TABLE estados (  
    id SERIAL PRIMARY KEY,  
    nome TEXT,  
    sigla TEXT  
);
```

Inserção de dados

```
INSERT INTO estados (nome, sigla)  
VALUES ('Paraná', 'PR'), ('São Paulo', 'SP');
```




Conceito

Uma chave estrangeira cria um relacionamento entre duas tabelas.

Exemplo

```
CREATE TABLE cidades (  
    id SERIAL PRIMARY KEY,  
    nome TEXT,  
    estado_id INTEGER,  
  
    FOREIGN KEY (estado_id)  
    REFERENCES estados(id)  
);
```

Inserção de dados

```
INSERT INTO cidades  
(nome, estado_id)  
VALUES  
('Curitiba', 1),  
('Campinas', 2);
```



Problema

Um aluno pode visitar vários estados.

Um estado pode ser visitado por vários alunos.

Exemplos

- João visitou PR, SP e RJ.
- Maria visitou SP e MG.
- O estado de São Paulo foi visitado por João e Maria.

Conclusão

Nenhuma das tabelas pode armazenar diretamente essa relação.

É necessário criar uma **tabela de associação**.

Aluno $\leftrightarrow$ Ja_Visitei $\leftrightarrow$ Estado



Tabela de alunos

```
CREATE TABLE alunos (  
    id SERIAL PRIMARY KEY, nome TEXT  
);
```

Tabela de estados (já existente)

Tabela N:M

```
CREATE TABLE ja_visitei (  
    aluno_id INTEGER,  
    estado_id INTEGER,  
  
    PRIMARY KEY (  
        aluno_id,  
        estado_id  
    ),  
    FOREIGN KEY (aluno_id)  
        REFERENCES alunos(id),  
    FOREIGN KEY (estado_id)  
        REFERENCES estados(id)  
);
```

Dados de exemplo

```
INSERT INTO alunos (nome)
VALUES
('João'),
('Maria');
```

```
INSERT INTO ja_visitei
(aluno_id, estado_id)
VALUES
(1, 1), -- João -> PR
(1, 2), -- João -> SP
(1, 3), -- João -> RJ
(2, 2), -- Maria -> SP
(2, 4); -- Maria -> MG
```

Quais estados João visitou?

```
SELECT
    a.nome,
    e.sigla
FROM alunos a
JOIN ja_visitei j
    ON a.id = j.aluno_id
JOIN estados e
    ON e.id = j.estado_id
WHERE a.nome = 'João';
```

Resultado: PR, SP e RJ.



1 Conceitos

2 Exercícios

3 Exercícios com EDGV

Criação da tabela

```
CREATE TABLE cidades (  
    id SERIAL PRIMARY KEY,  
    nome TEXT,  
    populacao INTEGER,  
    longitude REAL,  
    latitude REAL  
);
```

Carga dos dados

```
INSERT INTO cidades  
(nome, populacao, longitude, latitude)  
VALUES  
('Rio de Janeiro', 6748000, -43.1729, -22.9068),  
('São Paulo', 12330000, -46.6333, -23.5505),  
('Belo Horizonte', 2531000, -43.9386, -19.9208),  
('Campinas', 1214000, -47.0608, -22.9056);
```



Enunciado

A tabela possui colunas de longitude e latitude. Crie uma coluna espacial geom e preencha-a com geometrias do tipo ponto no sistema WGS84 (EPSG:4326).

1. Adicionar a coluna geom

```
ALTER TABLE cidades
ADD COLUMN geom geometry(Point, 4326);
```

2. Preencher a geometria a partir das coordenadas

```
UPDATE cidades
SET geom = ST_SetSRID(
    ST_MakePoint(
        longitude,
        latitude
    ),
    4326
);
```



Enunciado

Carregue o arquivo br.json na tabela

Enunciado

Carregue o arquivo br.json na tabela

```
import geopandas as gpd
from sqlalchemy import create_engine

# Ler o GeoJSON
gdf = gpd.read_file("br.json")

# Conexão com o PostGIS
engine = create_engine(
    "postgresql+psycopg://postgres:postgres@localhost/geodb"
)

# Criar/substituir tabela no banco
gdf.to_postgis(
    name="brasil",
    con=engine,
    if_exists="replace",
    index=True
)

print("Camada importada com sucesso.")
```



Enunciado

Liste todas as cidades cadastradas e mostre o campo geometria como texto.



Enunciado

Liste todas as cidades cadastradas e mostre o campo geometria como texto.

```
SELECT ST_AsText(geom), *  
FROM cidades;
```



Enunciado

Liste apenas o nome e a população das cidades.



Enunciado

Liste apenas o nome e a população das cidades.

```
SELECT nome, populacao  
FROM cidades;
```



Enunciado

Liste as cidades com população superior a 3 milhões de habitantes.



Enunciado

Liste as cidades com população superior a 3 milhões de habitantes.

```
SELECT *  
FROM cidades  
WHERE populacao > 3000000;
```



Enunciado

Ordene as cidades pela população, da maior para a menor.



Enunciado

Ordene as cidades pela população, da maior para a menor.

```
SELECT nome, populacao  
FROM cidades  
ORDER BY populacao DESC;
```



Enunciado

Determine quantas cidades estão cadastradas.



Enunciado

Determine quantas cidades estão cadastradas.

```
SELECT COUNT(*)  
FROM cidades;
```



Enunciado

Calcule a população média das cidades.



Enunciado

Calcule a população média das cidades.

```
SELECT AVG(populacao)
FROM cidades;
```



Enunciado

Determine a menor e a maior população cadastradas.



Enunciado

Determine a menor e a maior população cadastradas.

```
SELECT
    MIN(populacao) AS menor,
    MAX(populacao) AS maior
FROM cidades;
```



Enunciado

Liste as cidades cujo nome começa com a letra "C".



Enunciado

Liste as cidades cujo nome começa com a letra "C".

```
SELECT *  
FROM cidades  
WHERE nome LIKE 'C%';
```



Enunciado

Ordene as cidades da mais oeste para a mais leste utilizando a longitude.



Enunciado

Ordene as cidades da mais oeste para a mais leste utilizando a longitude.

```
SELECT nome, longitude
FROM cidades
ORDER BY longitude;
```



Enunciado

Classifique as cidades em Pequena, Média ou Grande e conte quantas existem em cada categoria.



Enunciado

Classifique as cidades em Pequena, Média ou Grande e conte quantas existem em cada categoria.

```
SELECT
    CASE
        WHEN populacao < 2000000
            THEN 'Pequena'
        WHEN populacao < 5000000
            THEN 'Media'
        ELSE 'Grande'
    END AS categoria,
    COUNT(*) AS total
FROM cidades
GROUP BY categoria;
```



1 Conceitos

2 Exercícios

3 Exercícios com EDGV



Objetivo

Importar automaticamente todos os shapefiles contidos em um arquivo ZIP para o banco PostgreSQL/PostGIS.

```
from pathlib import Path
import tempfile
import zipfile

import geopandas as gpd
from sqlalchemy import create_engine

# conexão com o banco
engine = create_engine(
    "postgresql+psycopg2://"
    "postgres:postgres@localhost/geodb"
)

arquivo_zip = "dados.zip"

with tempfile.TemporaryDirectory() as tmpdir:
```



Enunciado

Faça uma consulta que verifique quais linhas de `hid_trecho_drenagem_l` estão dentro de polígonos de `hid_trecho_massa_dagua_a`.



Enunciado

Faça uma consulta que verifique quais linhas de hid_trecho_drenagem_l estão dentro de polígonos de hid_trecho_massa_dagua_a.

```
SELECT
    d.*
FROM hid_trecho_drenagem_l d
JOIN hid_trecho_massa_dagua_a m
ON ST_Within(d.geom, m.geom);
```